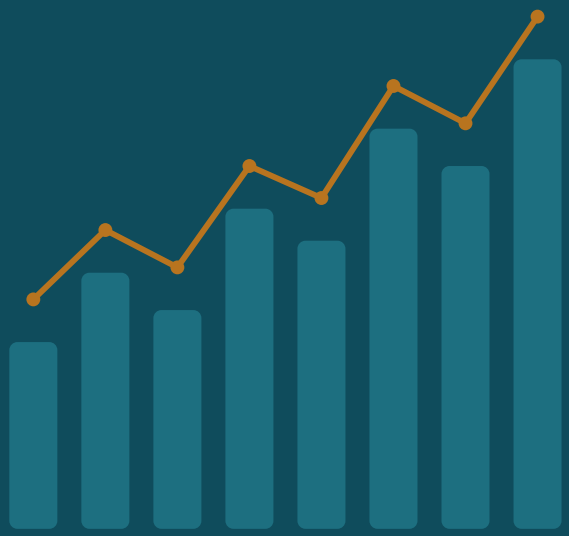
SOLUTION BLUEPRINT · VERSION 1.0

VK Finance

Family budget management platform on Salesforce

Plan, architecture, security design, flow diagrams and a 16-week build roadmap for the VKFinance org (Developer Edition · Experience Cloud · React)



Unlimited
families, each fully isolated

5 roles
admin, co-admin, member, child, viewer

10
flow & architecture diagrams

16 weeks
phased plan to v1.0

PREPARED FOR	VKFinance project owner (platform owner and Family 1 admin)
TARGET ORG	VKFinance · Developer Edition · API 68.0 · checked live on 27 Oct 2026
DATE	27 October 2026
STATUS	Design baseline - ready to start Phase 0

Confidential - personal project documentation

Contents

1 Executive summary . . . 4
2 Your VKFinance org - what we checked . . . 4
3 Goals, personas and scope . . . 5
3.1 Goals . . . 5
3.2 Personas and roles . . . 5
3.3 Scope . . . 6
4 Key architecture decisions . . . 6
5 Solution architecture . . . 7
5.1 How each layer behaves . . . 7
6 Multi-family tenancy and isolation . . . 8
6.1 The eight isolation rules . . . 8
6.2 Role permission matrix . . . 9
7 Data model . . . 10
7.1 Tenancy and identity . . . 11
7.2 Ledger . . . 11
7.3 Commitments . . . 12
7.4 Planning and insight . . . 12
8 Session management . . . 13
8.1 Session policy (Security_Policy__mdt) . . . 13
8.2 Client behaviour (React) . . . 13
9 User management . . . 16
9.1 Username and step-up rules . . . 16
10 Password management . . . 17
10.1 How passwords are stored . . . 17
10.2 Password policy . . . 17
11 Security management . . . 19
11.1 Defence in depth . . . 19
11.2 Threats and mitigations . . . 19
12 Finance management . . . 20
12.1 Accounts . . . 20
12.2 Categories (default template) . . . 20
12.3 Transactions . . . 21
12.4 Bank transfers . . . 22
12.5 Credit cards . . . 22

12.6 Bills . . . 23
12.7 Loans and EMI . . . 23
12.8 Budgets, recurring entries and goals . . . 23
13 Dashboard management . . . 24
13.1 Dashboard controls . . . 24
14 Report management . . . 25
14.1 Report features . . . 25
15 Notifications and automation . . . 26
16 Platform owner console (internal) . . . 27
17 Technical build blueprint . . . 27
17.1 REST API - /services/apexrest/vkf/v1 . . . 27
17.2 Apex inventory . . . 28
17.3 React app layout (uiBundles/VKFinancePortal/src) . . . 28
17.4 Project structure . . . 29
17.5 Code samples . . . 29
18 Developer Edition constraints . . . 31
19 Testing and quality . . . 32
20 Delivery roadmap . . . 33
21 Risks and mitigations . . . 34
22 Next steps - your first ten tasks . . . 34

1 Executive summary

VK Finance is a multi-family budgeting platform built on your VKFinance Salesforce org. As platform owner you onboard any number of families. Each family has a **Family Admin** who manages their own members (spouse, children, parents). Every member signs in to one Experience Cloud site and records earnings, expenses, bank transfers, credit-card spend, bills and EMIs. They see dashboards and reports limited to what their role allows. **No family can ever see another family's data.**

Key decisions at a glance

- **Experience Cloud + React.** The family portal is a React app (Salesforce Multi-Framework UI bundle) hosted on an Experience Cloud site. This org supports it today. Section 2 shows what was checked.
- **Your own user object, as requested.** Members are `Portal_User__c` records, not Salesforce users. They reach the site as guests, and every data call carries a server-validated VK session token. This allows unlimited families and members, with no per-user licence.
- **Family__c is the tenant boundary.** Every business record carries a `Family__c` lookup. The family comes from the session on the server, never from the browser.
- **One security gateway.** Every portal call passes through SessionService, then AccessGuard, then FamilyScope before it touches data. The guest profile has no object permissions at all.
- **One ledger.** Every money movement is a `Transaction__c` row. Transfers and card-bill payments are two linked rows, so card spending is never counted twice.
- **Built for Developer Edition limits** (5 MB data, 15 emails/day): rolling EMI windows, monthly snapshots, audit in a Big Object, retention jobs, in-app notifications first.

What this document gives you:

- Architecture, tenancy model and data model (sections 4–7).
- Session, user, password and security management designs, each with a flow diagram (sections 8–11).
- The finance modules: accounts, categories, transactions, transfers, credit cards, bills, loans and EMI, budgets (section 12).
- Dashboard, report and notification management (sections 13–15), and the platform-owner console (section 16).
- A concrete build blueprint: REST API, Apex classes, React app layout and code samples (section 17).
- Developer Edition constraints, testing, a 16-week roadmap, risks and your first ten tasks (sections 18–22).

2 Your VKFinance org - what we checked

These facts were read live from the connected org (alias `VKFinance`, user vkfinance@agentforce.com) with the Salesforce CLI on 27 October 2026. Only read-only queries were run, and nothing was deployed.

Check	Result	What it means for VK Finance
Edition and release	Developer Edition · instance USA1336 · API 68.0	Current release. Multi-Framework (React) needs Summer '26 or later, so this org qualifies.
Experience Cloud	Digital Experiences enabled · 0 sites yet · 11 templates incl. **Build Your Own (LWR)**	You can create the portal site now. No existing site to migrate or clean up.
React support	`UIBundle` metadata type present · CLI has `template generate ui-bundle` (reactbasic) and `template generate project -t reactexternalapp` · `ui-bundle dev` for local preview	**Yes, React builds are supported.** The reactexternalapp template hosts a React bundle on an Experience Cloud site and calls guest-accessible Apex REST. That is the exact pattern this design uses.
Local toolchain	Salesforce CLI 2.149.9 · Node 24.15 · npm 11.12	Meets the Multi-Framework requirements (CLI 2.130.7+, Node 22+).

Check	Result	What it means for VK Finance
Experience Cloud licences	External Apps Login **40** · Customer Community 5 · Customer Community Login 5 · Customer Community Plus 5 · Plus Login 5 · Partner Community 5 · Partner Login 5 · External Identity 9	You do have real licences, roughly 40–75 external users in total. The custom identity in this design is still the route to *unlimited* families. See the option note below.
Storage	Data **5 MB** · File 20 MB	About 2,500 records in total at about 2 KB each. This drives the storage-saving design choices in section 18.
Daily limits	Single email 15 · Mass email 10 · API requests 15,000 · Async Apex 250,000	Email can't be the main alert channel. Keep REST calls lean: one call per page for the dashboard.

Option note - you have licences after all

You chose a custom user object because you expected no Experience Cloud licences. This org actually has 40 External Apps Login licences plus several community licences. Both routes run on the same Experience Cloud site:

- **A. Custom identity (this design, as you asked):** unlimited families and members, and full control. You build and own login, sessions, passwords, lockout and MFA.
- **B. Native Experience Cloud users:** Salesforce provides login, password policy, MFA and sessions, and sharing sets give family isolation. You are capped by the licence count, and every member needs a Contact and a User.

The design keeps identity behind one Apex interface (`IdentityProvider`). You can start with A and switch some or all families to B later without rewriting the finance modules. Licensing: the guest + custom-login pattern is fine for a personal Developer Edition project. If this ever moves to a paid production org, use real Experience Cloud licences, because guest access for named users can breach the licence terms.

3 Goals, personas and scope

3.1 Goals

- One place for a family's money: earnings, expenses, bank and wallet accounts, credit cards, bills, loans and EMIs, budgets and goals.
- Every family member has their own login and sees only what their role allows.
- The Family Admin sees everything inside their own family: every member's transactions, dashboards and reports.
- Unlimited families, strictly isolated from each other, managed by you as platform owner.
- Strong session, user, password, security and audit controls even though identity is custom-built.

3.2 Personas and roles

Persona	Typical person	Works in	Can do
Platform owner	You (Salesforce System Administrator)	Internal Lightning app	Create, suspend or delete families; appoint Family Admins; unlock or reset anyone; view audit and platform health; manage policies and default categories.
Family Admin	Head of a family (you, for Family 1)	React portal	Everything inside their own family: members and roles, accounts, budgets, bills, loans, every member's transactions and reports, member security.
Co-Admin	Spouse or partner	React portal	Same as Family Admin, except they can't remove or demote the Family Admin or change family settings.
Member	Adult son or daughter, working parent	React portal	Own transactions and accounts, plus accounts shared with them. Can read family budgets and see their own reports.
Child	School or college-age child	React portal	Record own expenses and pocket money and see own budget. Expenses above a limit can optionally need approval.
Viewer	Grandparents or other read-only relatives	React portal	Read-only dashboards the Family Admin grants: own data or family level.

3.3 Scope

In v1.0 (weeks 1–16)	Later (Phase 6 backlog)
Multi-family tenancy · custom login, sessions, passwords, lockout · member management · accounts (bank, cash, UPI wallet, credit card, loan, FD) · two-level categories · income, expense, transfer and card-payment transactions · bills and reminders · loans with EMI schedule and payments · card statements and utilisation · budgets with alerts · recurring entries · configurable dashboards · 13 reports with CSV export · audit trail · in-app notifications · admin MFA (email OTP or authenticator app)	Bank-statement CSV import with matching rules · receipt photos · child spend requests and pocket-money automation · goal auto-contributions · multi-currency · installable mobile app (PWA) · AI spending insights · native-licence identity option for selected families

4 Key architecture decisions

#	Decision	Choice	Why	Trade-off
1	Portal UI	React 19 UI bundle (Salesforce Multi-Framework) on an Experience Cloud site	You asked for React, the org supports it, and it suits a dashboard-heavy app (rich npm chart libraries, local dev with `ui-bundle dev`)	Pages are managed in code, not Experience Builder. Newer stack (GA since Summer '26) with fewer community examples.
2	Identity	Custom `Portal_User__c` + `Portal_Session__c`; members reach the site as guests	Unlimited members, and it works without per-user licences	You own login security - designed in sections 8–11.
3	Data access from React	Guest-accessible Apex REST `/vkf/v1/*` via `sdk.fetch`, token in `X-VKF-Session` header	Apex is the only door to data. A header token also makes CSRF a non-issue.	GraphQL / UI API is **not** used for finance data (the guest user has no object access by design).
4	Tenant boundary	Required, indexed `Family__c` lookup on every business object	Simple, selective queries; scales to many families	Must be enforced in code - mitigated by one selector base class and isolation tests.
5	Apex layering	REST resource (with sharing) → service (inherited sharing) → selector (without sharing, family-filtered)	Guest users can't own or see records through sharing, so elevated access is contained in one auditable layer	A selector bug is high impact, so it gets a code-review rule and tests.
6	Ledger	Single `Transaction__c`; transfers and card payments are two linked rows	Correct balances; card spend counted once	Slightly more logic in TransferService.
7	Balances	Stored on `Financial_Account__c`, updated by triggers, re-verified nightly	Fast dashboards, self-healing	Needs a reconcile batch.
8	Portal analytics	Apex aggregate SOQL + Recharts; `Monthly_Summary__c` snapshots	Family-scoped by construction; cheap trends	No drag-and-drop report builder for members - saved filter presets instead.
9	Configuration	Custom Metadata: Security_Policy, Role_Permission, Default_Category	Deployable and versioned; tune without code	Edited in Setup or by deploy.
10	Audit	`Audit_Event__e` (publish immediately) → `Audit_Log__b` big object	Survives rollbacks, so failed logins still get logged; doesn't use the 5 MB data storage	Big-object queries must filter by index (family, time).
11	Notifications	In-app `Notification__c` first, email optional	Only 15 single emails per day	Users see alerts when they open the portal.

5 Solution architecture

The solution has two front doors: the React portal for family members, and the internal Lightning app for you. Both lead to one Apex layer and one data model. Figure 1 shows the layers, and Figure 5 in section 8 follows a single request through them.

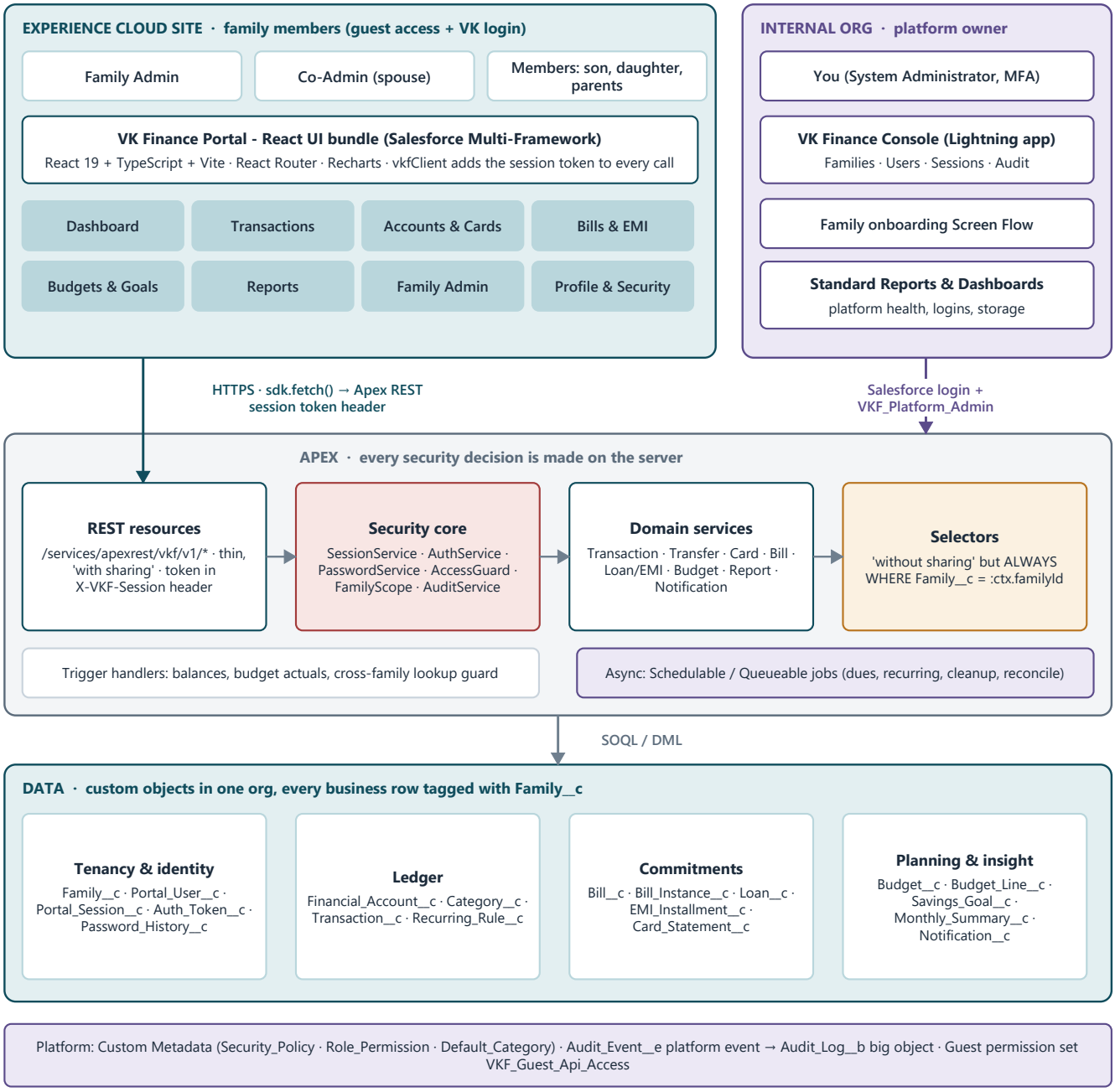


Figure 1 - Solution architecture: React portal on Experience Cloud, Apex security gateway, family-tagged data

5.1 How each layer behaves

- **Experience Cloud site.** Created from the `reactexternalapp` scaffold. It is an LWR app-container site with public access enabled, and it serves the React bundle (`ui-bundle.json` routes every path to `index.html`). Self-registration is off, guest Chatter is off, and families are onboarded only by you.
- **React portal.** Vite + TypeScript + React Router + Tailwind/shadcn (from the template), plus Recharts for charts. One `vkfClient` wraps `sdk.fetch`: it adds the `X-VKF-Session` header and handles 401 centrally. The router hides private pages without a token, but the real enforcement is always on the server.
- **Apex REST resources.** Thin `@RestResource` classes under `/vkf/v1/`, one per feature area. They parse input, call the security gateway, call a service, and map the result or error to JSON and an HTTP status.

- **Security core.** SessionService validates tokens, AuthService and PasswordService handle credentials, AccessGuard applies the role matrix, FamilyScope enforces tenancy, and AuditService publishes events.
- **Domain services and selectors.** Business rules live in services. Every query lives in a selector that extends `FamilyScopedSelector`, which always applies `Family__c = :ctx.familyId`.
- **Internal org.** You sign in with normal Salesforce login and MFA. Standard record pages, a Screen Flow for onboarding, a few LWCs (audit viewer, session monitor), and standard Reports and Dashboards for platform health.

> **Why not GraphQL / UI API for finance data?**
> The template's `graphqlClient.ts` uses the Data SDK GraphQL client. That runs as the current Salesforce user with sharing and field-level security. Your members are guests, and the guest profile deliberately has no access to any VK object. Granting it would expose every family's data to every visitor. So finance data flows only through Apex REST endpoints that check the VK session first. GraphQL stays available for the internal console, where you are a real Salesforce user.

6 Multi-family tenancy and isolation

Family 1 can't see Family 2, and a Family Admin controls only their own family's members. Figure 2 shows the model. Because the portal runs through the guest user, Salesforce sharing can't separate families for us, so isolation is enforced in Apex by eight rules that every endpoint follows.

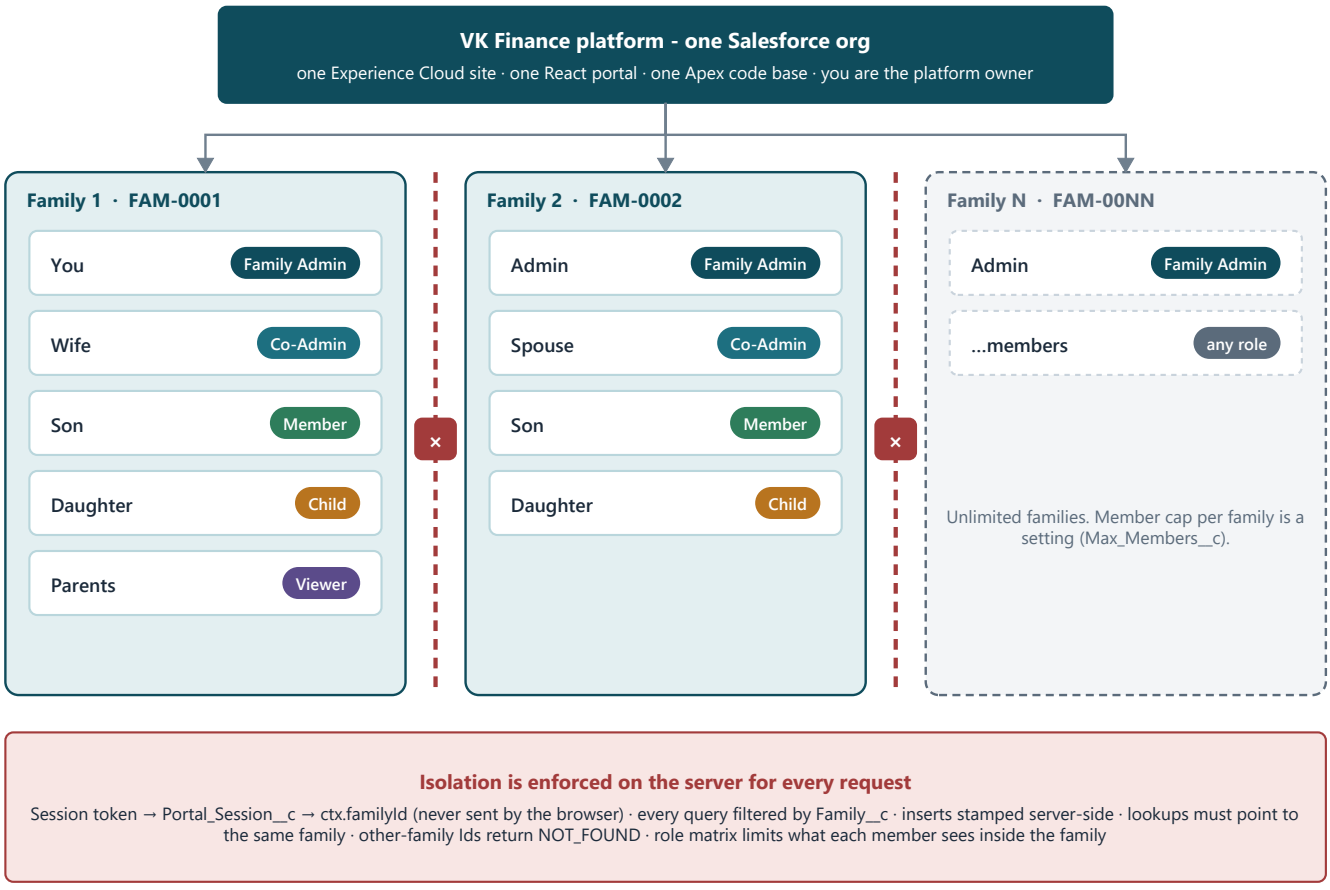


Figure 2 - Tenant model: unlimited families, each walled off on the server

6.1 The eight isolation rules

1. **Family is never a request parameter.** Endpoints accept a session token, not a familyId. FamilyScope reads `ctx.familyId` from the validated session.
2. **Every business object has** `Family__c` (lookup, required, indexed). On insert it is stamped server-side, overwriting anything the client sent.
3. **Every portal query lives in a selector** that extends `FamilyScopedSelector` and always filters by family. A CI check fails the build if a REST or service class contains inline SOQL.

4. **Update and delete by Id re-query first** with the family filter. A missing record returns **NOT_FOUND**, never "forbidden", so Ids from other families can't be probed.
5. **Cross-reference guard:** a transaction's account, category, member, bill and loan must all belong to the same family. This is checked in the trigger handler, so bulk loads and internal edits are protected too.
6. **Member scope inside a family:** `ctx.visibleMemberIds()` returns all members for Family Admin and Co-Admin, self plus shared accounts for Member and Child, and whatever is granted for Viewer.
7. **Suspending a family** revokes all of its sessions immediately and refuses new logins.
8. **Automated isolation tests** for every endpoint: a Family B user aiming at Family A Ids must get NOT_FOUND. See section 19.

6.2 Role permission matrix

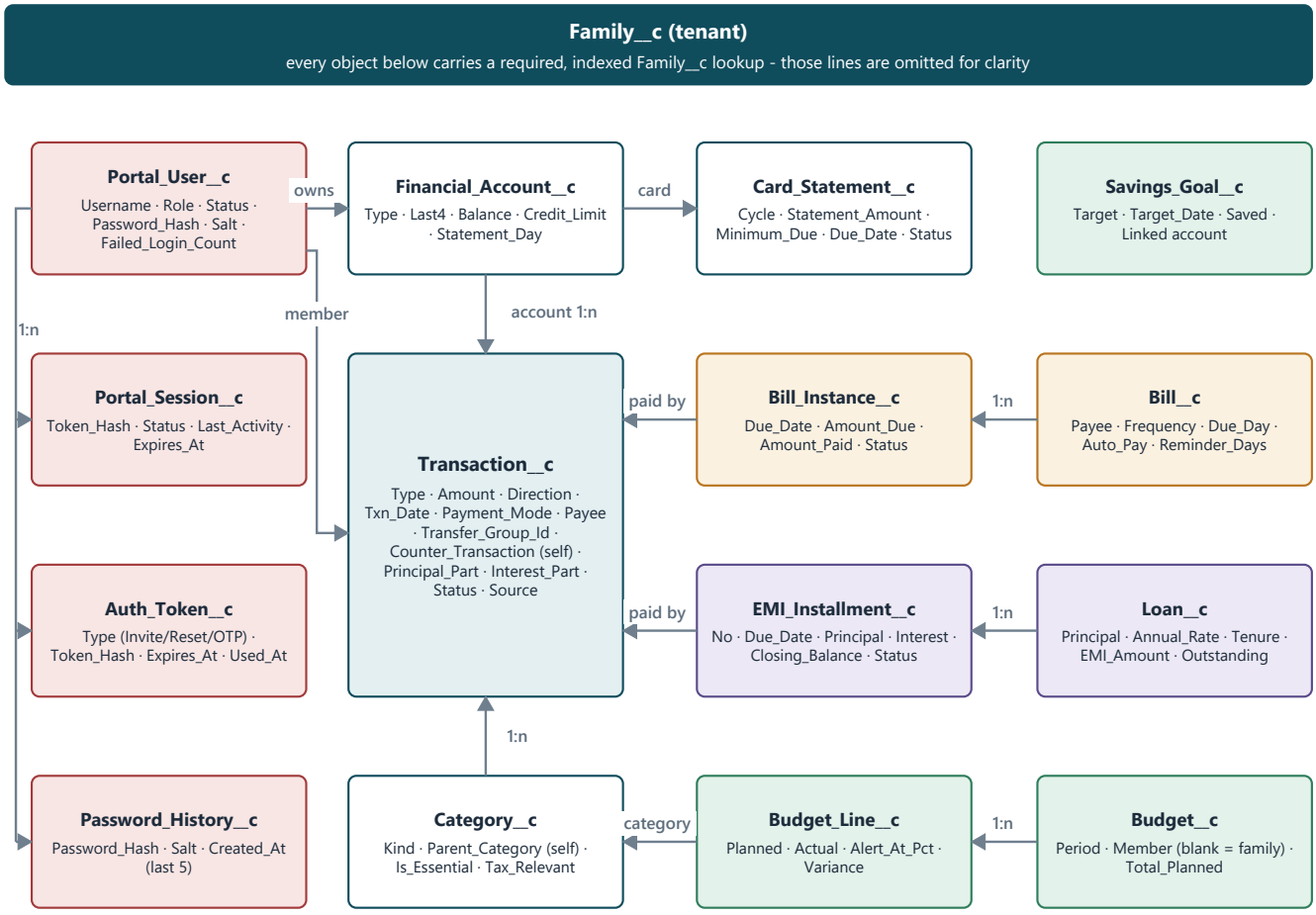
Stored as `Role_Permission__mdt` rows (role × feature × action × scope) and cached by AccessGuard. Legend: **Full** = create, read, edit and delete across the family · **Read** = read family-wide · **Own** = own records only · **Own+S** = own plus shared accounts · **-** = no access.

Feature	Family Admin	Co-Admin	Member	Child	Viewer
Family dashboard	Full	Full	Own+S	Own	Read*
Own transactions	Full	Full	Full	Own (create, edit within 24 h)	-
Other members' transactions	Full	Full	-	-	Read*
Accounts and cards	Full	Full	Own+S	Read own	-
Bills	Full	Full	Read + pay assigned	-	-
Loans and EMI	Full	Full	Read own	-	-
Budgets	Full	Full	Read	Read own	Read*
Reports	All members	All members	Own	Own	As granted
Member management	Full	Full (not the Family Admin)	-	-	-
Security of others (sessions, unlock, reset)	Full	Full (not the Family Admin)	Own	Own	Own
Family settings and categories	Full	Read	Read	-	-
Family audit trail	Read	Read	-	-	-

** The Family Admin grants Viewer access per viewer with `Viewer_Scope__c` (Own or Family). The Co-Admin can never demote, remove, reset or unlock the Family Admin. A family always keeps at least one Family Admin.*

7 Data model

Twenty custom objects in five groups, plus three Custom Metadata types, a platform event and a big object. Figure 3 shows the relationships that matter. Every business object also looks up to `Family__c`, but those lines are left out for readability.



Supporting objects

Recurring_Rule__c (salary, SIP, rent, subscriptions) · Monthly_Summary__c (family / member / month snapshot for fast trends) · Notification__c (in-app bell) · Saved_Report__c (filter presets) · Dashboard layout JSON on Portal_User__c

Platform & configuration

Security_Policy__mdt (timeouts, lockout, password rules, hash iterations) · Role_Permission__mdt (role × feature × action × scope) · Default_Category__mdt (cloned into each new family) · Audit_Event__e (platform event, publish immediately) → Audit_Log__b (big object indexed by Family__c + Event_Time__c - keeps the 5 MB data storage free)

Figure 3 - Entity relationships (Family__c lookups omitted)

7.1 Tenancy and identity

Object	Purpose	Key fields
Family__c	The tenant. One row per family.	Family_Code__c (unique, ext. id, e.g. FAM-0001) · Status__c (Active / Suspended) · Base_Currency__c (INR) · Time_Zone__c · Month_Start_Day__c · Max_Members__c · Large_Txn_Threshold__c · Primary_Admin__c
Portal_User__c	A person who can sign in (your custom "user").	Family__c · First/Last_Name__c · Username__c (unique, case-insensitive) · Email__c · Mobile__c · Relationship__c (Self, Spouse, Son, Daughter, Father, Mother, Other) · Role__c · Status__c (Invited, Active, Locked, Disabled, Removed) · Password_Hash__c · Password_Salt__c · Hash_Iterations__c · Password_Changed_At__c · Must_Change_Password__c · Failed_Login_Count__c · Locked_Until__c · Last_Login__c · Monthly_Spend_Limit__c · Viewer_Scope__c · Dashboard_Layout__c (JSON) · MFA_Method__c
Portal_Session__c	One signed-in browser session.	Portal_User__c · Family__c · Token_Hash__c (unique, ext. id) · Status__c (Active, Logged Out, Expired, Revoked) · Scope__c (FULL / CHANGE_PASSWORD) · Last_Activity__c · Expires_At__c · User_Agent_Hash__c · Revoked_By__c · Revoke_Reason__c
Auth_Token__c	One-time tokens.	Portal_User__c · Type__c (Invite, Reset, Email OTP) · Token_Hash__c · Expires_At__c · Used_At__c · Attempts__c
Password_History__c	Stops password reuse.	Portal_User__c · Password_Hash__c · Password_Salt__c · Created_At__c (last 5 kept)

7.2 Ledger

Object	Purpose	Key fields
Financial_Account__c	Anything that holds or owes money.	Owner_Member__c · Type__c (Savings, Current, Cash, UPI Wallet, Credit Card, Loan, Fixed Deposit, Investment) · Institution__c · Last4__c (never full numbers) · Opening_Balance__c · Current_Balance__c · Is_Liability__c (formula) · Credit_Limit__c · Statement_Day__c · Payment_Due_Days__c · Min_Due_Pct__c · Is_Shared__c · Status__c
Category__c	Two-level income and expense classification.	Name · Kind__c (Income, Expense, System) · Parent_Category__c (self) · Is_Essential__c (need vs want) · Tax_Relevant__c · Icon__c · Color__c · Sort_Order__c · Active__c
Transaction__c	The ledger: every money movement.	Member__c · Account__c · Category__c · Type__c (Income, Expense, Transfer Out, Transfer In, Card Payment, Bill Payment, EMI Payment, Refund, Adjustment) · Amount__c (always positive) · Direction__c (Debit / Credit) · Txn_Date__c · Payment_Mode__c (Cash, UPI, Debit Card, Credit Card, NEFT/IMPS, Cheque, Auto-debit) · Payee__c · Notes__c · Reference_No__c · Transfer_Group_Id__c · Counter_Transaction__c · Bill_Instance__c · EMI_Installment__c · Principal_Part__c · Interest_Part__c · Status__c (Pending, Cleared, Void) · Source__c (Manual, Recurring, Auto-pay, Import)
Recurring_Rule__c	Template for repeating entries.	Member__c · Type__c · Amount__c · Account__c · Category__c · Frequency__c · Next_Run_Date__c · End_Date__c · Mode__c (Auto-post / Suggest)

7.3 Commitments

Object	Purpose	Key fields
Bill__c	A recurring biller: electricity, internet, school fees, insurance premium.	Category__c · Payee__c · Consumer_No__c · Frequency__c · Due_Day__c · Expected_Amount__c · Pay_From_Account__c · Auto_Pay__c · Reminder_Days__c · Responsible_Member__c · Active__c
Bill_Instance__c	One due occurrence of a bill.	Bill__c (master-detail) · Due_Date__c · Amount_Due__c · Amount_Paid__c · Status__c (Upcoming, Due, Overdue, Partially Paid, Paid, Skipped) · Paid_On__c
Loan__c	Home, car, personal, education, gold, consumer-durable or card-EMI loan.	Borrower_Member__c · Lender__c · Loan_Type__c · Principal__c · Annual_Rate__c · Tenure_Months__c · Start_Date__c · EMI_Day__c · EMI_Amount__c · Outstanding_Principal__c · Debit_Account__c · Loan_Account__c · Auto_Debit__c · Status__c
EMI_Installment__c	Amortisation row (next 12 kept as records).	Loan__c (master-detail) · Installment_No__c · Due_Date__c · EMI_Amount__c · Principal_Part__c · Interest_Part__c · Opening_Balance__c · Closing_Balance__c · Status__c · Paid_On__c · Late_Fee__c
Card_Statement__c	Monthly credit-card bill.	Account__c (card) · Cycle_Start__c · Cycle_End__c · Statement_Amount__c · Minimum_Due__c · Due_Date__c · Amount_Paid__c · Status__c

7.4 Planning and insight

Object	Purpose	Key fields
Budget__c	A period's plan for the family or for one member.	Member__c (blank = family) · Period_Start__c · Period_End__c · Total_Planned__c · Total_Actual__c · Status__c
Budget_Line__c	Planned vs actual per category.	Budget__c (master-detail) · Category__c · Planned__c · Actual__c · Alert_At_Pct__c (default 80) · Variance__c (formula) · Rollover__c
Savings_Goal__c	Vacation, emergency fund, education.	Member__c · Target_Amount__c · Target_Date__c · Saved_Amount__c · Linked_Account__c
Monthly_Summary__c	Snapshot per family, member and month.	Member__c · Month__c · Income__c · Expense__c · Savings__c · By_Category__c (JSON)
Notification__c	In-app alerts.	Recipient__c · Type__c · Message__c · Link__c · Is_Read__c · Expires_On__c
Saved_Report__c	Saved report filter presets.	Owner__c · Report_Type__c · Filters__c (JSON) · Shared_With_Family__c

Modelling rules

- Business objects use a **required lookup** to `Family__c`, not master-detail. That keeps the two master-detail slots free (Bill → Instance, Loan → Installment, Budget → Line) and lets family deletion run as a controlled batch.
- Secret fields (`Password_Hash__c`, `Password_Salt__c`, `Token_Hash__c`) are on no page layout and readable by no profile. Only system-mode Apex touches them, and they are never put in a DTO.
- Card and account numbers: store **last 4 digits only**. Never store a full card number, CVV, PIN or net-banking password.
- Turn on field history tracking for `Portal_User__c.Role__c` and `Status__c`, and for `Family__c.Status__c`.

8 Session management

A VK session is a `Portal_Session__c` row identified by the SHA-256 hash of a 256-bit random token. The browser holds the raw token; the database never does. Figure 4 (end of this section) shows how a session is created, and Figure 5 shows how every later request is checked.

8.1 Session policy (Security_Policy__mdt)

Setting	Default	Notes
Token	256-bit random, hex	`Crypto.generateAesKey(256)`. Only `SHA-256(token)` is stored, so a leaked database export can't be replayed.
Idle timeout	30 minutes	React shows a 'Stay signed in?' dialog 2 minutes before expiry and calls `GET /auth/session` to extend.
Absolute lifetime	12 hours	Re-login required after this, however active the user is.
Concurrent sessions	3 per user	The oldest is revoked when a 4th starts.
Activity writes	At most once per minute	Saves DML and storage. Last_Activity is approximate to within 60 s.
Binding	User-agent hash	A token replayed from a different browser is rejected.
Revoked automatically on	Password change, reset, role change, disable, family suspend	Except the session that made a self-service password change.
Restricted scope	CHANGE_PASSWORD	Issued when a temporary password is used. Only `/auth/first-login` accepts it.
Clean-up	Hourly job	Marks expired sessions; deletes closed ones after 7 days (storage).

8.2 Client behaviour (React)

- The token lives in `sessionStorage`, so it is per tab and cleared when the browser closes. A 'Keep me signed in on this device' option is deliberately left out of v1.
- Apex REST can't set HttpOnly cookies, so the token is visible to scripts on the page. Mitigations: no third-party scripts (everything is bundled at build time), strict CSP, React's default output escaping, short idle timeout, user-agent binding and revoke-all.
- `vkfClient` handles 401 in one place: clear the token, keep the current route as a safe relative `next` parameter, and go to `/login`.
- **Sign out** calls `POST /auth/logout` (status Logged Out). **Sign out everywhere** revokes all of the user's sessions.
- Family Admins can see each member's active sessions (device, started, last active) and revoke them.

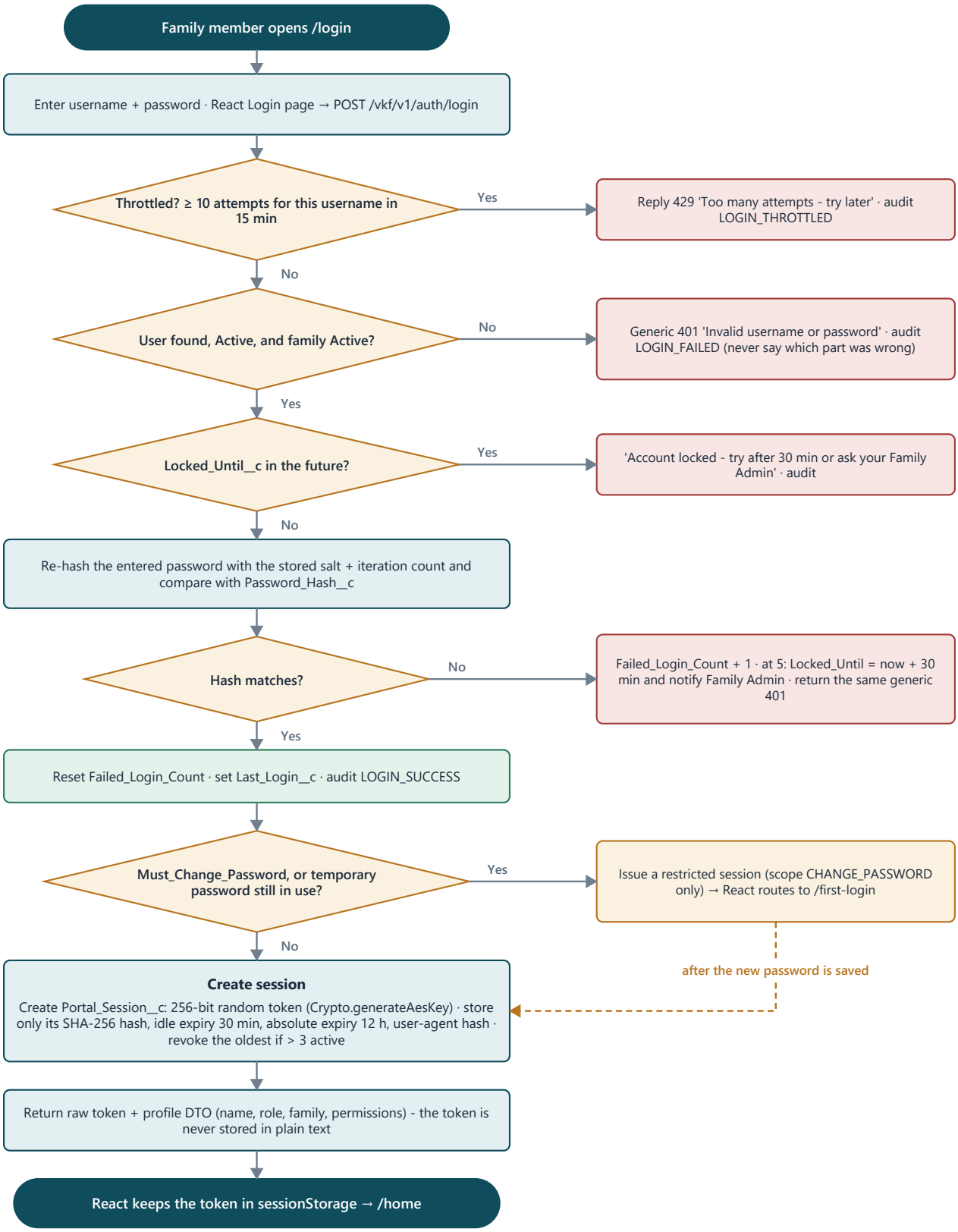


Figure 4 - Login and session creation

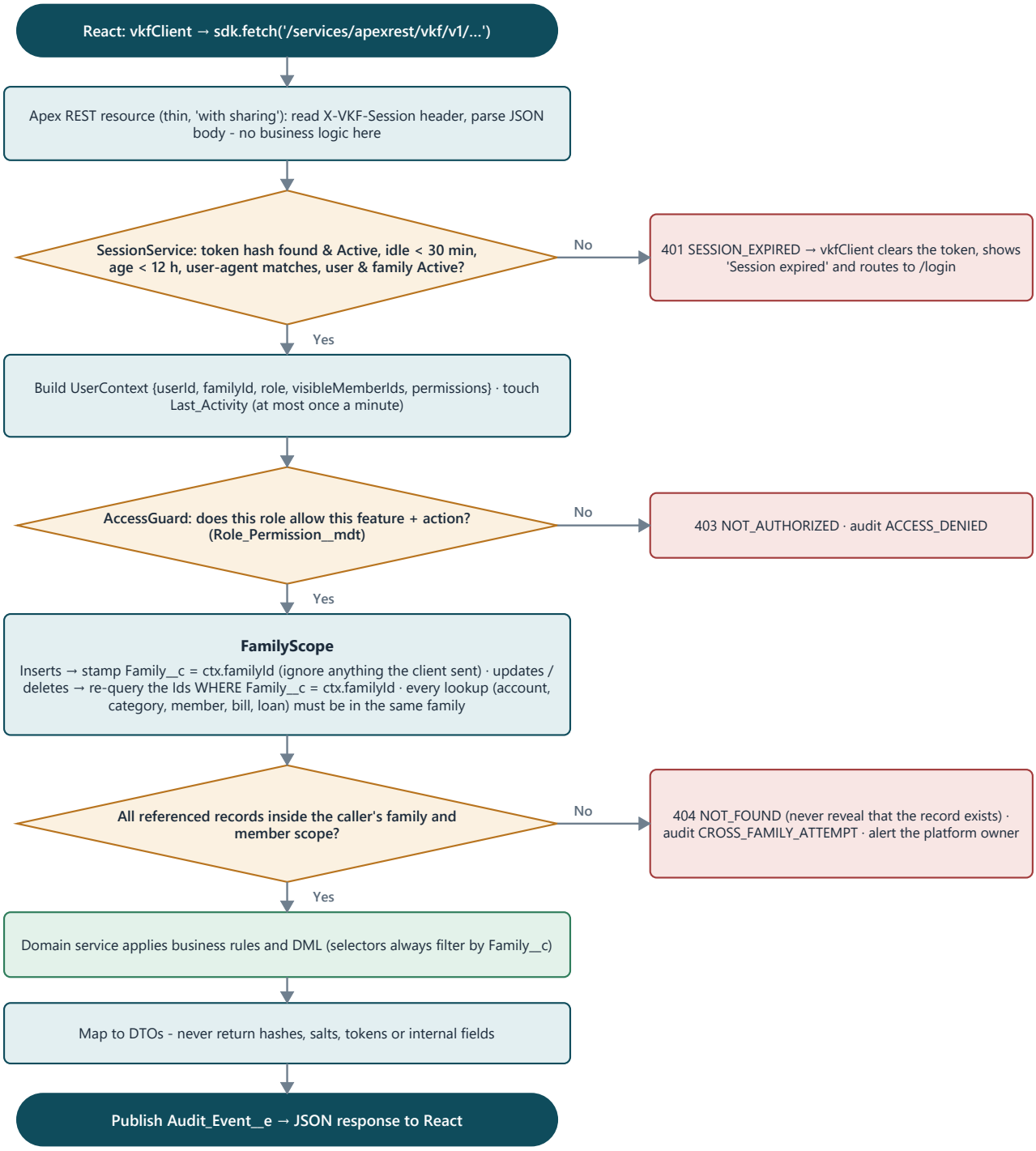


Figure 5 - Per-request security pipeline (every portal endpoint)

9 User management

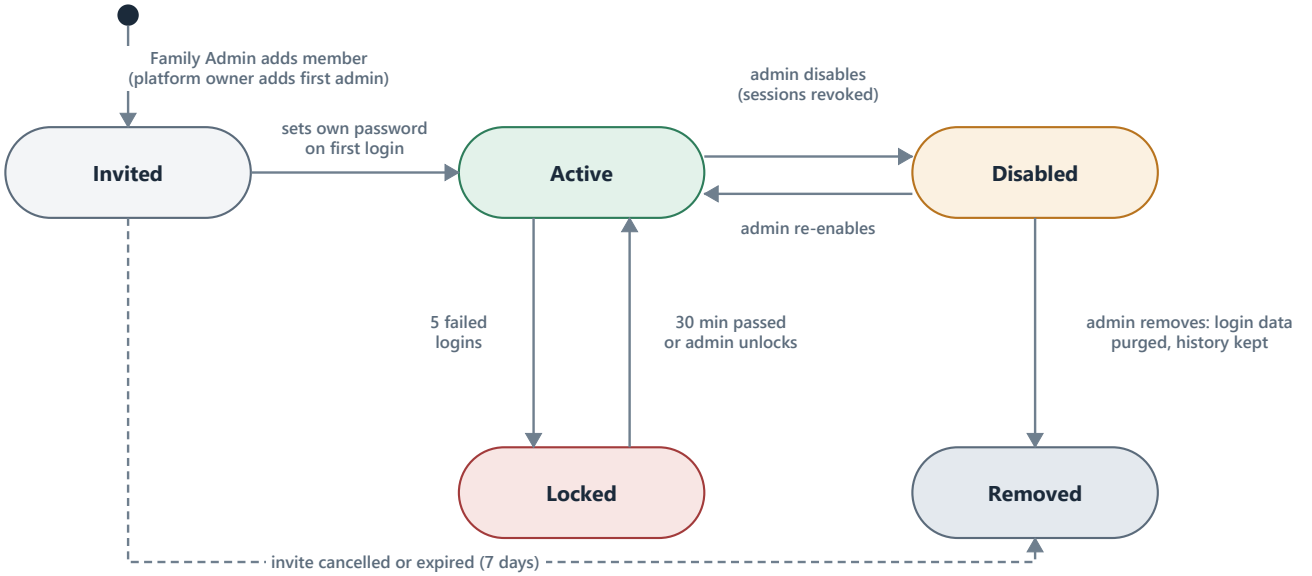


Figure 6 - Portal user lifecycle

Action	Who	What happens
Create family + first admin	Platform owner (internal Screen Flow)	Family__c created with the next Family_Code__c; Default_Category__mdt cloned; Portal_User__c (Family Admin, Invited) created; temporary password shown once or invite link emailed.
Add member	Family Admin, Co-Admin	Checks Max_Members__c and username uniqueness; role chosen (only a Family Admin can create another Family Admin); invite or temporary password.
Change role	Family Admin	All of that member's sessions revoked; audit ROLE_CHANGED; at least one Family Admin must remain.
Disable / re-enable	Family Admin, Co-Admin	Disable revokes sessions; data and history stay.
Unlock	Family Admin, Co-Admin, platform owner	Clears Locked_Until__c and the failed-login counter.
Reset password	Family Admin, Co-Admin, platform owner	Temporary password (24 h) and Must_Change_Password__c - see Figure 7.
Transfer admin rights	Family Admin	Promote another adult to Family Admin, then step down; requires password re-entry.
Remove member	Family Admin	Status Removed; hash, salt, sessions and tokens purged; transactions kept and shown as 'former member'.
Edit own profile	Everyone	Name, email, mobile, avatar, dashboard layout - never role or status.
Spending limits	Family Admin	Monthly_Spend_Limit__c per member; alerts when crossed.

9.1 Username and step-up rules

- Usernames are **globally unique across all families**, so sign-in needs only username + password. 4–40 characters (letters, digits, dot, dash, underscore), case-insensitive. Email is optional for children.
- **Step-up authentication:** sensitive admin actions (role change, admin transfer, resetting someone else's password, family deletion request) require the admin to re-enter their password, plus their OTP once MFA is on.
- Invites expire after 7 days. Expired invites move to Removed automatically.

10 Password management

10.1 How passwords are stored

- **Key stretching:** iterated HMAC-SHA256 (PBKDF2-style) with `Crypto.generateMac`, a per-user 128-bit random salt and an iteration count from Security_Policy__mdt. Benchmark in the org and choose the highest count that keeps login under about 1 second of CPU.
- Algorithm and iteration count are stored per user. When you raise the count later, a user is re-hashed on their next successful login.
- Never: reversible encryption, logging passwords (watch debug logs), or returning hashes or salts to the client.
- An optional pepper can live in a protected custom setting. In an org where you are the only admin it guards against data exports, not against admins.

```
public inherited sharing class PasswordService {
    public static String newSalt() {
        return EncodingUtil.base64Encode(Crypto.generateAesKey(128));
    }

    public static String hash(String password, String salt, Integer iterations) {
        Blob key = Blob.valueOf(password);
        Blob digest = Crypto.generateMac('hmacSHA256', EncodingUtil.base64Decode(salt), key);
        for (Integer i = 1; i < iterations; i++) {
            digest = Crypto.generateMac('hmacSHA256', digest, key);
        }
        return EncodingUtil.base64Encode(digest);
    }
}
```

10.2 Password policy

Rule	Default
Minimum length	10 characters (12 for Family Admin and Co-Admin)
Composition	At least 3 of: upper case, lower case, digit, symbol
Blocked	Contains username, first name or family code; on the common-password list (static resource)
History	Can't reuse the last 5
Expiry	Off by default (NIST SP 800-63B); optional 180 days
Lockout	5 failures → 30 minutes; repeat lockouts double the wait
Temporary password	12 random characters, valid 24 h, must be changed at first use
Reset link	Single use, 30 minutes, only the hash stored
Change password	Requires the current password; revokes all other sessions

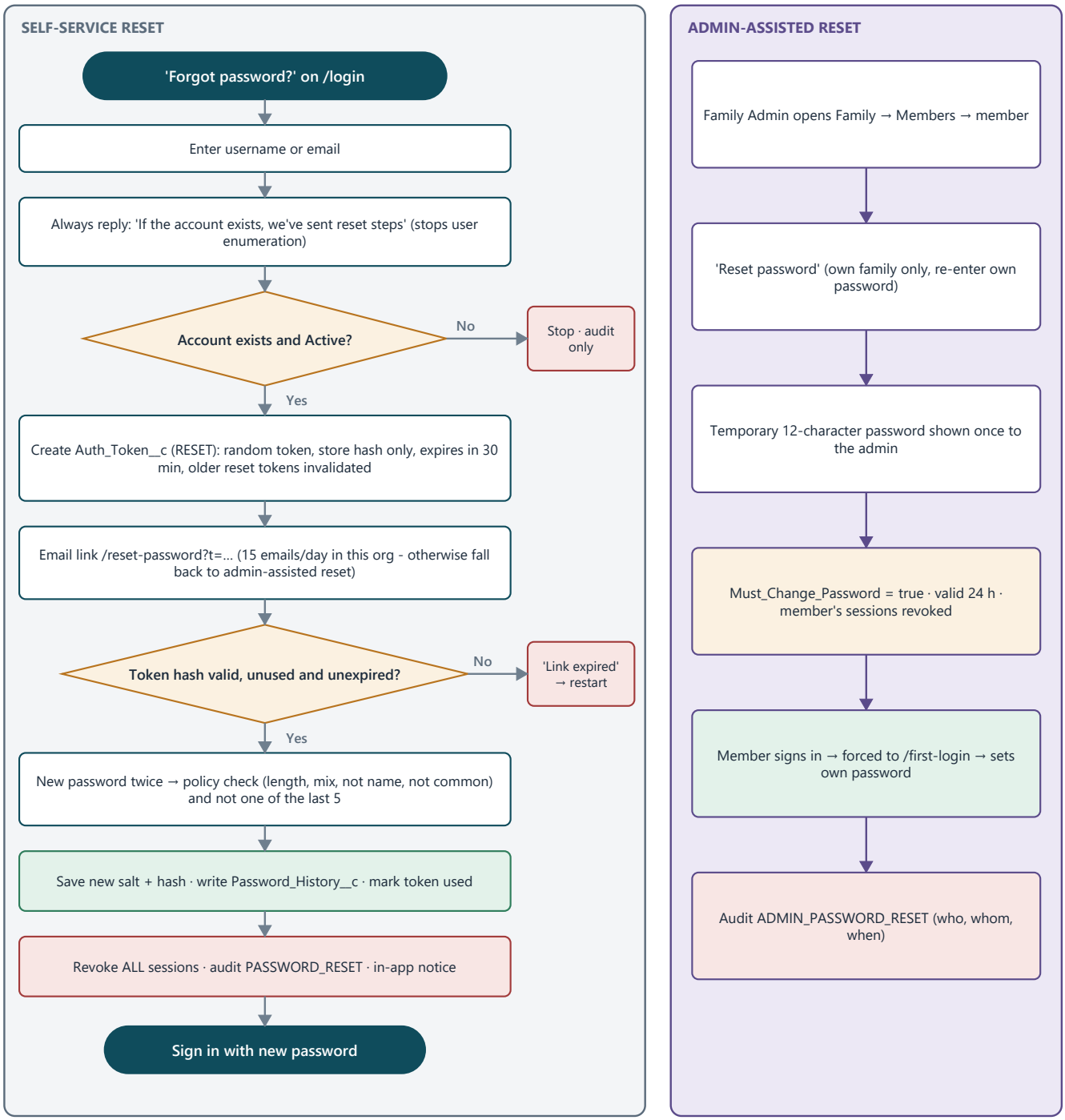


Figure 7 - Password reset: self-service and admin-assisted

11 Security management

11.1 Defence in depth

Layer	Controls
Platform	My Domain, HTTPS only, MFA on your Salesforce login, Health Check score ≥ 90%, OWD Private on every VK object, VKF_Platform_Admin permission set held only by you
Site / guest user	Guest profile: **no object permissions** on any VK object · permission set VKF_Guest_Api_Access: API Enabled + Apex class access to the `Vkf*Resource` classes only · self-registration off · guest Chatter off (the template turns it on) · guest file access off · members can't see each other · clickjack protection Same Origin · no third-party CSP sites
Identity	Iterated salted hashing, lockout, per-username throttling, generic errors, short sessions, step-up authentication, optional admin MFA (email OTP or authenticator app via `Auth.SessionManagement` TOTP helpers)
Authorisation	AccessGuard role matrix on every endpoint · FamilyScope tenancy · member scope · role and family always read from the database, never from the request
Data	Last-4 digits only · no CVV, PIN or banking passwords ever · secret fields hidden from every profile · DTO whitelists (no mass assignment) · Classic Encrypted Text for the TOTP secret
Code	Bind variables only · whitelisted sort and filter fields for any dynamic SOQL · error codes, not stack traces · Salesforce Code Analyzer (`sf code-analyzer run`) + ESLint + Prettier in the Husky pre-commit hook already in this project
Monitoring	Audit trail for every login and every create, update and delete · alerts for lockouts, cross-family attempts and new admins · weekly security digest to you

11.2 Threats and mitigations

Threat	Example	Mitigation
Brute force / credential stuffing	Thousands of passwords tried against 'dad'	Per-username throttle (429), lockout after 5, identical generic errors, optional OTP for admins
Session hijack	Token copied from a shared laptop	sessionStorage, 30-minute idle, 12-hour cap, user-agent binding, sign out everywhere, revoke on password change
IDOR / cross-family access	Editing a record Id in a request	FamilyScope re-query, NOT_FOUND, isolation tests, audit + alert to you
Privilege escalation	Child calls an admin endpoint from the console	AccessGuard on every endpoint; role comes from the database
Mass assignment	Client sends Family__c or Role__c in a JSON body	Request DTO whitelists; server stamps tenant and ownership fields
SOQL injection	Malicious sort parameter	Bind variables, whitelists, `String.escapeSingleQuotes` for any LIKE search
XSS	Script typed into payee or notes	React escapes output; `dangerouslySetInnerHTML` banned by lint rule; strict CSP
User enumeration	Reset form reveals which usernames exist	Same response and same code path for unknown users
Open redirect	Crafted `next` after login	Only relative paths accepted (the template's `isValidRedirect` already does this)
Data leak via caching	CDN caches a guest response	All `/vkf/v1` responses send `Cache-Control: no-store`; nothing private is in the static bundle

12 Finance management

12.1 Accounts

Type	Balance means	Notes
Savings / Current	Money you have (asset)	Opening balance + credits − debits; reconcile with an Adjustment entry
Cash	Cash in hand (asset)	One per member is typical
UPI wallet	Wallet balance (asset)	UPI payments straight from a bank account use that bank account
Credit card	Outstanding you owe (liability)	Credit_Limit__c, Statement_Day__c, Payment_Due_Days__c, Min_Due_Pct__c
Loan	Principal outstanding (liability)	Created automatically with each Loan__c
Fixed deposit / Investment	Current value (asset)	Updated manually or by recurring rule

Net worth = assets − liabilities, shown on the dashboard for the family (admins) or for the member's own and shared accounts.

12.2 Categories (default template)

Cloned from `Default_Category__mdt` into each new family. Families can rename, hide or add categories. **Is_Essential** drives the needs-vs-wants split, and **Tax_Relevant** feeds the annual summary.

Category	Sub-categories
Income	Salary · Bonus and Incentives · Business / Freelance · Rental Income · Interest and Dividends · Refunds and Cashback · Gifts Received · Other Income
Food and Dining	Groceries · Restaurants and Takeaway · Snacks and Beverages · Milk and Daily Essentials
Outing and Entertainment	Movies and Shows · Trips and Picnics · Events and Functions · Hobbies and Sports
Healthcare	Doctor Consultation · Hospital · Lab Tests and Scans · Dental and Eye Care · Health Insurance Premium
Medicine	Pharmacy · Supplements and Vitamins · Medical Devices
Housing	Rent · Society Maintenance · Repairs and Services · Property Tax · Furnishing
Utilities	Electricity · Water · Cooking Gas · Internet · Mobile Recharge · DTH / Cable
Transport	Fuel · Cab / Auto · Public Transport · Vehicle Service · Parking and Tolls · Vehicle Insurance
Education	School / College Fees · Tuition and Coaching · Books and Stationery · Online Courses
Shopping	Clothing and Footwear · Electronics · Home and Kitchen · Accessories
Personal Care	Salon and Grooming · Cosmetics · Fitness and Gym
Kids	Pocket Money · Toys and Games · Activities and Classes
Parents and Elder Care	Support Money · Caregiver · Medical for Parents
Insurance	Life / Term · Home
Subscriptions	OTT / Streaming · Apps and Software · Newspapers and Magazines
Travel and Vacation	Flights and Trains · Hotels · Sightseeing
Gifts and Donations	Gifts Given · Charity · Religious Offerings
EMI and Loans	Loan Interest · Processing Fees
Taxes and Fees	Income Tax · Bank Charges · Late Fees and Penalties
Pets · Miscellaneous	Food · Vet · Grooming · Uncategorised
System (not income or expense)	Transfer · Credit Card Payment · Loan Principal · Opening Balance / Adjustment

12.3 Transactions

Type	Rows	Account effect	Counted as	Example
Income	1	Asset ↑	Income	Salary ₹85,000 credited to savings
Expense	1	Asset ↓, or card outstanding ↑	Expense in its category	Groceries ₹3,200 by UPI; dinner ₹2,400 on a card
Refund / Cashback	1	Asset ↑ or card ↓	Negative expense in the original category	₹500 refund for returned shoes
Transfer	2 linked	Source ↓, destination ↑	Neither	₹10,000 savings → wife's account
Card payment	2 linked	Bank ↓, card outstanding ↓	Neither - spend was counted at swipe	₹15,000 card bill paid from savings
Bill payment	1 + bill update	Asset ↓ (or card ↑)	Expense in the bill's category	Electricity ₹1,850
EMI payment	1 + installment + loan update	Bank ↓, loan outstanding ↓ by principal	Interest = expense; principal = liability reduction; full EMI in the cash-flow view	EMI ₹20,758 = ₹13,258 principal + ₹7,500 interest
Adjustment	1	± to match the bank	Neither	Reconcile to a bank statement

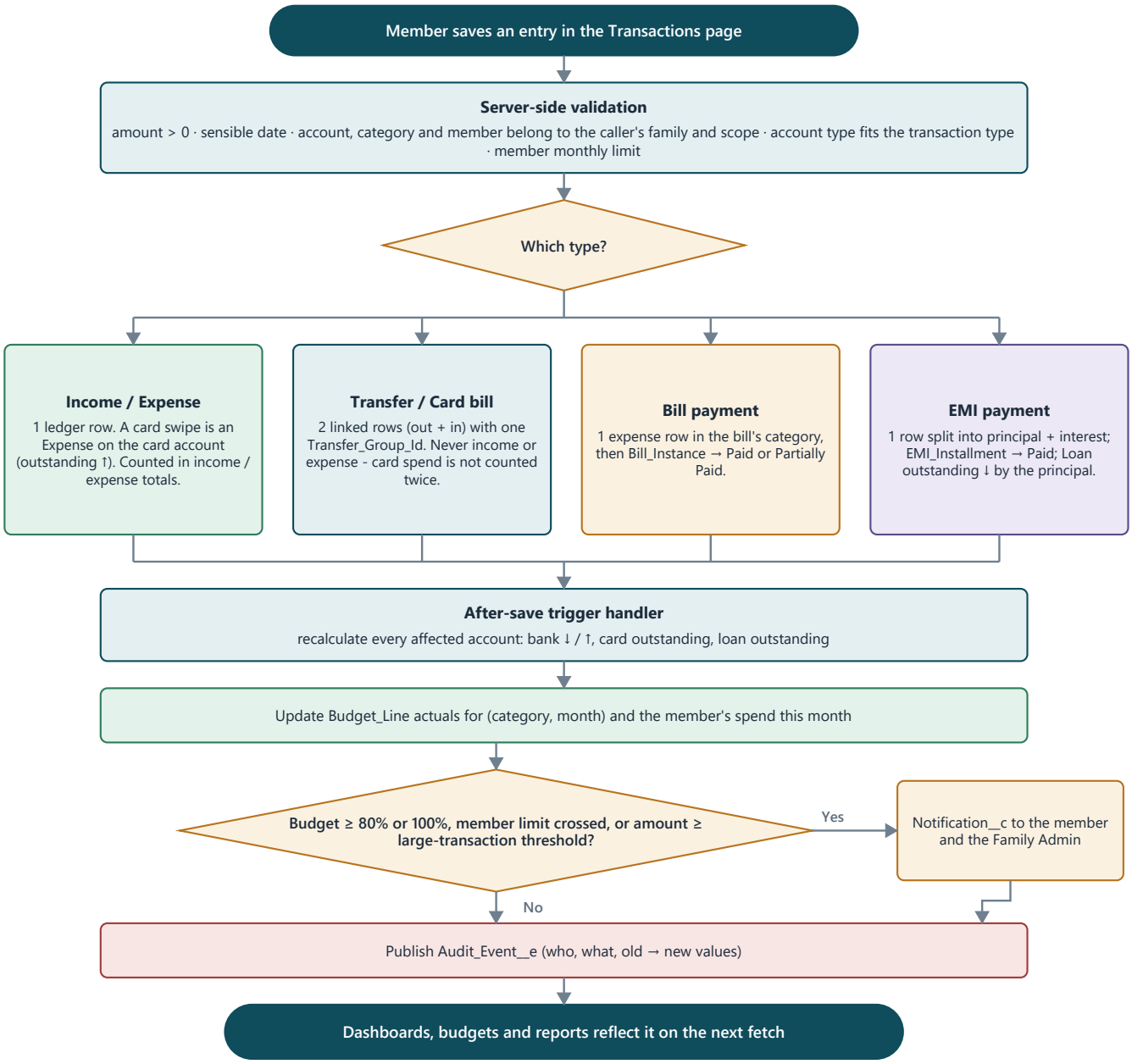


Figure 8 - Transaction posting: one ledger, four shapes, consistent side effects

12.4 Bank transfers

- **Between your own accounts** (savings → cash, savings → FD): one Transfer request creates both legs atomically with a shared `Transfer_Group_Id__c`. Family totals don't change.
- **To another family member's account** (pocket money, household money to spouse): same two legs. It shows in both members' views and stays neutral for the family.
- **To people outside the family** (rent to a landlord, payment to a plumber): an Expense with payment mode NEFT/IMPS/UPI and a payee, because the money leaves the family.
- Invariant checked in tests and nightly: within every transfer group, debits equal credits.

12.5 Credit cards

- **Usage:** every swipe is an Expense on the card account in the right category, so spending is counted when it happens.
- **Billing cycle:** from the day after the last statement to Statement_Day__c (clamped to month end for days 29–31).
- **Statement** = previous unpaid balance + card expenses − refunds − payments in the cycle. Fees and interest are entered as Taxes and Fees → Bank Charges. Minimum due = max(Min_Due_Pct × statement, floor). Due date =

statement date + Payment_Due_Days__c.

- **Payment:** a Card Payment transfer (bank → card). Paid in full → Paid; less → Partially Paid; nothing by the due date → Overdue.
- **Utilisation** = outstanding ÷ limit. An advisory appears above 30%.
- **Card EMI conversion:** turns a big purchase into a Loan__c (type Card EMI, loan account = the card). Each instalment is added to the card statement instead of debiting the bank.

12.6 Bills

- Frequency: Monthly, Bi-monthly, Quarterly, Half-yearly, Yearly or One-time. Each has a due day, an expected amount (the actual amount is entered when paying), a responsible member and reminder days.
- DailyDuesJob creates the next Bill_Instance, sends reminders and flags Overdue (Figure 9).
- **Mark paid** opens a pre-filled payment. **Auto-pay** bills post themselves on the due date from the configured account.
- Bill history shows on-time vs late payments and the annual cost per biller.

12.7 Loans and EMI

EMI = $P \times r \times (1 + r)^n \div ((1 + r)^n - 1)$, where r = annual rate ÷ 12 ÷ 100 and n = months. Worked example: ₹10,00,000 at 9% for 60 months gives an EMI of **₹20,758**. The first instalment splits into ₹7,500 interest and ₹13,258 principal, and total interest over the loan is about ₹2,45,501.

- On creation the EMI is computed. If the lender's figure differs slightly, store the lender's EMI and use it.
- The full schedule is computed on the fly for display (`AmortizationCalculator`). Only the **next 12** instalments are kept as `EMI_Installment__c` records - a rolling window that protects the 5 MB limit.
- **Pay EMI** creates an EMI Payment (principal and interest split), marks the instalment Paid and reduces Outstanding_Principal__c.
- **Prepayment** is a principal-only payment. Then choose reduce-tenure or reduce-EMI, and the remaining schedule is regenerated. Foreclosure closes the loan.
- Loan report: outstanding, % repaid, payoff date, and interest and principal paid this financial year (useful for tax declarations such as home-loan interest).

12.8 Budgets, recurring entries and goals

- **Budgets:** monthly by category, for the family and optionally per member. 'Copy last month' and 'use 3-month average' fill a new budget. Optional rollover of unspent amounts. Alerts at 80% and 100%. A special **EMIs** line covers full EMI outflow, because families plan in cash-flow terms.
- **Needs vs wants:** Is_Essential__c drives a 50/30/20-style view (needs / wants / savings).
- **Recurring rules:** salary on the 1st, rent on the 3rd, SIP on the 5th, streaming subscriptions. Mode Auto-post creates the transaction; Suggest creates a Pending entry to confirm.
- **Savings goals:** target and date. Monthly contribution needed = (target − saved) ÷ months left. Progress follows the linked account's balance.

13 Dashboard management

The dashboard is one `GET /vkf/v1/dashboard?period=…&member=…` call that returns every widget's data at once. That keeps API usage low and the page fast. Each widget is a React component fed from that single payload.

Widget	Shows	Default for
Month at a glance	Income, expense, net savings, savings rate vs last month	Everyone
Budget health	Planned vs actual bars per category, coloured at 80% and 100%	Admins, members, child (own)
Spending by category	Donut + top 5 sub-categories, click to drill down	Everyone (scoped)
6-month trend	Income vs expense lines, savings bars	Admins, members
Upcoming dues (15 days)	Bills, EMIs and card due dates with amounts and status	Admins, members
Accounts and net worth	Balances by account, assets − liabilities	Admins; members (own + shared)
Credit-card health	Outstanding, utilisation gauge, next due date	Admins, card holders
Loans	Outstanding principal, % repaid, payoff date	Admins
Member spending	Bar per member this month vs their limit	Family Admin, Co-Admin only
Recent transactions	Last 10 with quick edit	Everyone
Goals	Progress rings	Everyone
Alerts	Budget breaches, overdue bills, lockouts (admins)	Everyone

13.1 Dashboard controls

- **Per-user layout:** show, hide and reorder widgets. Stored as JSON in `Portal_User__c.Dashboard_Layout__c`.
- **Family defaults:** the Family Admin sets a default layout per role and can lock widgets (for example, children can't hide Budget health).
- **Filters:** period (this month, last month, quarter, year to date, custom) and account.
- **Member selector (admins only):** 'All members' or any one member. This is how a Family Admin sees each member's dashboard exactly as that member does, plus the family-wide view.

14 Report management

Report	Answers	Visual	Available to
Monthly family summary	Where did this month's money go?	KPI tiles + table	Admins (all members); others (own)
Spending by category	Which categories cost most? Drill to sub-category, then transactions	Donut + table	Everyone (scoped)
Member-wise spending	Who spent what, vs their limit?	Bar + table	Family Admin, Co-Admin
Income vs expense trend	Are we saving more over time?	12-month line	Everyone (scoped)
Budget vs actual	Which budgets are over?	Bullet bars	Everyone (scoped)
Cash-flow forecast	Will we have enough in 30 / 60 / 90 days?	Line	Admins
Account statement	Running balance for one account	Table	Owner, admins
Credit-card report	Usage by card and category, utilisation, statements, payments	Bar + table	Card holder, admins
Loans and EMI	Outstanding, paid vs remaining, interest this year, schedule	Stacked bar + table	Borrower, admins
Bills	Paid on time vs late, upcoming, annual cost per biller	Table + bar	Admins
Payment modes	Cash vs UPI vs card share	Donut	Everyone (scoped)
Annual summary	Year totals; tax-relevant categories (medical, education, insurance, home-loan interest)	Table	Admins
Security and activity	Logins, failures, lockouts, active sessions, admin actions	Table	Admins (family); platform owner (all)

14.1 Report features

- **Filters:** period, member (admins), account, category, type, payment mode, tags. Drill-down from category to sub-category to transactions.
- **Saved reports:** filter presets in `Saved_Report__c`, private or shared with the family.
- **Export:** CSV generated in the browser from the returned rows, plus a print-friendly view (browser 'Save as PDF').
- **Performance:** aggregate SOQL (`GROUP BY` with the family filter) for the current period, `Monthly_Summary__c` for anything older than 3 months, and pages of 200 rows for transaction lists.
- **Platform reports (internal):** standard Salesforce reports on Family__c, Portal_User__c and Portal_Session__c for families by status, active users, logins per day, lockouts and storage per family.

15 Notifications and automation

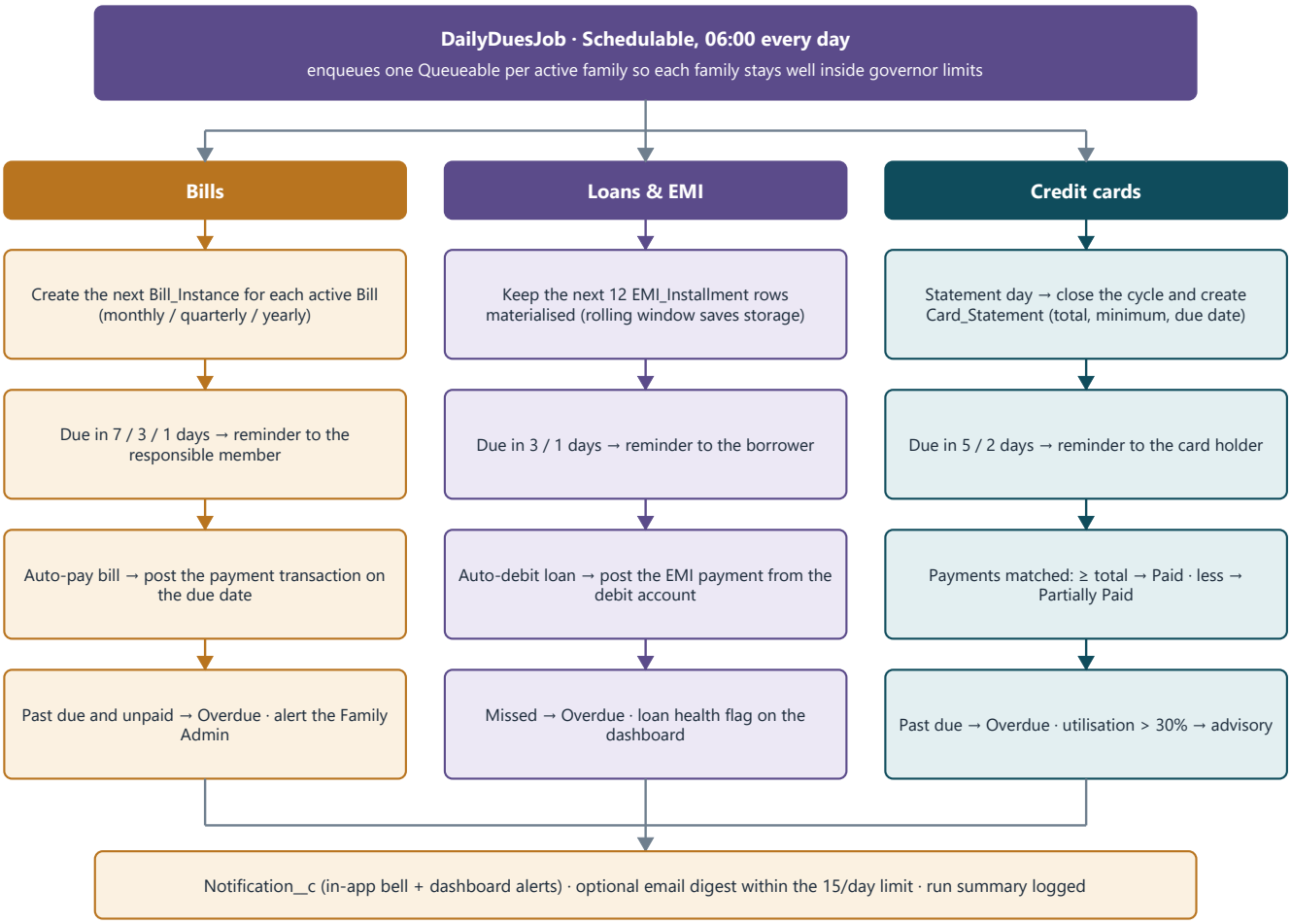


Figure 9 - Daily automation for bills, EMIs and credit cards

Job	Schedule	Purpose
DailyDuesJob → DuesForFamilyQueueable	06:00 daily	Bill instances, EMI window, card statements, reminders, overdue flags (Figure 9)
RecurringPostingJob	05:30 daily	Post or suggest due recurring transactions
SessionCleanupJob	Hourly	Expire idle and over-age sessions; delete closed sessions after 7 days; expired tokens
BalanceReconcileBatch	02:00 daily	Recompute balances and budget actuals from the ledger; log and fix drift
MonthlyCloseJob	1st of month, 03:00	Monthly_Summary__c snapshots; create next month's budgets from the template
RetentionPurgeJob	Sundays 04:00	Delete read notifications after 60 days, used tokens and password history beyond 5; storage report to you
SecurityDigestJob	Mondays 07:00	One email to you: lockouts, cross-family attempts, new admins, failed jobs

Notification types: bill, EMI and card due · overdue · budget 80% and 100% · member limit crossed · large transaction (admins) · account locked (admins) · new sign-in for admins · password changed. All are in-app (bell + dashboard). Email is reserved for password resets and your weekly digest, to stay within 15 emails a day.

16 Platform owner console (internal)

- **Lightning app 'VK Finance Console'** with tabs for Families, Portal Users, Sessions, Audit Viewer (LWC over Audit_Log__b), Security Policy, Default Categories and a Platform Dashboard.
- **Family onboarding Screen Flow:** (1) family name, currency, time zone, member cap → (2) first admin's name, username, email → (3) creates Family__c with the next code, clones categories, creates the admin as Invited, shows the temporary password once → (4) audit FAMILY_CREATED.
- **Family actions:** suspend (revokes all sessions, blocks login) · reactivate · export family data (CSV) · delete (batch delete after a 30-day grace period, confirmed by typing the family code).
- **Platform dashboard** (standard reports): families by status, users active in the last 30 days, logins per day, failed logins and lockouts, active sessions, records per family (storage), job failures.
- **Privacy:** as org owner you can technically read every family's data in Setup. The portal itself never crosses families. If other families need privacy from you, that is a trust and policy matter, so write it down for them.

17 Technical build blueprint

17.1 REST API - /services/apexrest/vkf/v1

Resource	Methods	Purpose	Access
/auth/login · /auth/logout · /auth/session	POST · POST · GET	Sign in, sign out, who-am-I + keep-alive	Public / any session
/auth/forgot · /auth/reset · /auth/first-login · /auth/password	POST · POST · POST · PUT	Self-service reset, set first password, change password	Public / restricted / any
/dashboard	GET	All widget data for a period and member	Per role
/transactions · /transactions/{id}	GET POST · PATCH DELETE	List (filters, paging), add, edit, void	Per role
/transfers	POST	Create both legs of a transfer or card payment	Per role
/accounts · /cards/{id}/statements	GET POST PATCH · GET	Accounts, card statements	Per role
/bills · /bills/instances/{id}/pay	GET POST PATCH · POST	Bills and payments	Admins; members pay assigned
/loans · /loans/{id}/schedule · /pay · /prepay	GET POST PATCH · GET · POST · POST	Loans and EMI	Admins; borrower reads
/budgets · /budgets/{id}/copy · /goals	GET POST PATCH · POST · GET POST PATCH	Planning	Per role
/reports/{type} · /reports/saved	GET · GET POST DELETE	Report data and presets	Per role
/family/members · .../{id}/reset · .../{id}/unlock · .../{id}/sessions	GET POST PATCH · POST · POST · GET DELETE	Member management and member security	Family Admin, Co-Admin
/family/settings · /family/categories · /family/audit	GET PATCH · GET POST PATCH · GET	Settings, categories, audit trail	Family Admin (Co-Admin reads)
/profile · /profile/sessions · /notifications	GET PATCH · GET DELETE · GET POST	Self-service	Any session

Error contract: 401 SESSION_EXPIRED · 403 NOT_AUTHORIZED · 404 NOT_FOUND · 409 CONFLICT · 422 VALIDATION (field errors) · 429 THROTTLED · 500 UNEXPECTED. The body is always `{"code": …, "errors": […]}`, with no stack traces. Assume every call counts against the org's 15,000 daily API requests, so pages make one or two calls.

17.2 Apex inventory

Layer	Classes
REST resources	VkfAuthResource · VkfDashboardResource · VkfTransactionResource · VkfTransferResource · VkfAccountResource · VkfCardResource · VkfBillResource · VkfLoanResource · VkfBudgetResource · VkfReportResource · VkfFamilyResource · VkfProfileResource · VkfNotificationResource · VkfRest (JSON + error mapping helper)
Security core	IdentityProvider (interface) + CustomIdentityProvider · AuthService · SessionService · PasswordService · TokenService · Throttle · UserContext · AccessGuard · FamilyScope · AuditService · SecurityPolicy · PortalException
Domain services	FamilyOnboardingService · MemberService · AccountService · CategoryService · TransactionService · TransferService · CardService · CardStatementService · BillService · LoanService · AmortizationCalculator · BudgetService · RecurringService · GoalService · NotificationService · DashboardService · ReportService
Selectors	FamilyScopedSelector (abstract) · PortalUserSelector · SessionSelector · AccountSelector · CategorySelector · TransactionSelector · BillSelector · LoanSelector · BudgetSelector · ReportSelector
Triggers	One trigger per object → TriggerHandler base · TransactionTriggerHandler (cross-family guard, balances, budget actuals, alerts) · BillInstanceTriggerHandler · EmiInstallmentTriggerHandler · PortalUserTriggerHandler (username normalisation, last-admin guard) · AuditEventTrigger
Async	DailyDuesJob · DuesForFamilyQueueable · RecurringPostingJob · SessionCleanupJob · BalanceReconcileBatch · MonthlyCloseJob · RetentionPurgeJob · SecurityDigestJob
Tests	TestDataFactory (two families, every role) · FamilyIsolationTest · SessionServiceTest · PasswordServiceTest · AuthServiceTest · AmortizationCalculatorTest · TransferServiceTest · CardStatementServiceTest · BudgetServiceTest · one test class per REST resource

17.3 React app layout (uiBundles/VKFinancePortal/src)

Folder	Contents
api/	vkfClient.ts (sdk.fetch wrapper, token header, 401 handling) · typed endpoint modules (transactions.ts, dashboard.ts, …) · shared DTO types
features/auth	Login · FirstLogin · ForgotPassword · ResetPassword · ChangePassword · OtpPrompt · SessionTimeoutDialog · useIdleTimer · AuthContext (token, profile, permissions)
features/dashboard	Dashboard · KpiTiles · BudgetHealth · CategoryDonut · TrendChart · UpcomingDues · AccountBalances · CardHealth · MemberSpend · RecentTransactions · GoalsWidget · LayoutEditor
features/money	Transactions (list, form, transfer form) · Accounts · CardDetail · CardStatement · Bills · Loans (form, schedule) · Budgets · RecurringRules · Goals
features/reports	ReportCenter · ReportFilters · ReportViewer · SavedReports · csvExport.ts
features/family	Members · MemberForm · MemberSecurity (sessions, unlock, reset) · FamilySettings · CategoryManager · AuditTrail
components/ui	shadcn/ui primitives from the template (button, card, table, dialog, tabs, …) + Money, PeriodPicker, MemberPicker, EmptyState
routes.tsx	/login · /forgot-password · /reset-password · /first-login · /home · /transactions · /accounts · /cards/:id · /bills · /loans/:id · /budgets · /goals · /reports/:type · /family · /profile · /notifications

17.4 Project structure

```
VKFinance/
  force-app/main/default/
    uiBundles/VKFinancePortal/      React app (Vite, TypeScript) - built to dist/ before deploy
    digitalExperiences/site/VK_Finance1/   Experience Cloud site (app container for the bundle)
    digitalExperienceConfigs/  networks/  sites/   site configuration
    permissionsets/                 VKF_Guest_Api_Access, VKF_Platform_Admin
    objects/                        Family__c, Portal_User__c, Transaction__c, ... + Audit_Event__e, Audit_Log__b
    customMetadata/                 Security_Policy.Default, Role_Permission.*, Default_Category.*
    classes/
      rest/  security/  services/  selectors/  triggers/  jobs/  tests/
    triggers/                       one trigger per object
    lwc/                            internal console components only (audit viewer, session monitor)
    flows/                          Family_Onboarding
    applications/                   VK_Finance_Console
    staticresources/                common_passwords
  docs/                             this blueprint
  scripts/apex/                     seed data for two demo families
```

17.5 Code samples

REST resource - thin, every method starts at the gateway

```
@RestResource(urlMapping='/vkf/v1/transactions/*')
global with sharing class VkfTransactionResource {

    @HttpGet
    global static void listTransactions() {
        RestRequest req = RestContext.request;
        try {
            UserContext ctx = SessionService.requireSession(VkfRest.sessionToken(req));
            AccessGuard.require(ctx, 'TRANSACTIONS', 'READ');
            Date fromDate = Date.valueOf(req.params.get('from'));
            Date toDate = Date.valueOf(req.params.get('to'));
            VkfRest.ok(TransactionService.listForPeriod(ctx, fromDate, toDate));
        } catch (Exception e) {
            VkfRest.fail(e);   // PortalException code -> 401/403/404/422; anything else -> generic 500
        }
    }
}
```

Session validation

```
public without sharing class SessionService {
    public static UserContext requireSession(String rawToken) {
        if (String.isBlank(rawToken)) {
            throw new PortalException('SESSION_EXPIRED');
        }
        String tokenHash = EncodingUtil.convertToHex(
            Crypto.generateDigest('SHA-256', Blob.valueOf(rawToken)));
        List<Portal_Session__c> rows = [
            SELECT Id, Family__c, Portal_User__c, Portal_User__r.Role__c, Portal_User__r.Status__c,
                   Family__r.Status__c, Scope__c, Last_Activity__c, Expires_At__c
            FROM Portal_Session__c
            WHERE Token_Hash__c = :tokenHash AND Status__c = 'Active'
            LIMIT 1
        ];
        Datetime now = Datetime.now();
        Integer idleMinutes = SecurityPolicy.current().Idle_Timeout_Minutes__c.intValue();
        // No DML before throwing: the exception rolls it back. SessionCleanupJob marks expiry.
        if (rows.isEmpty()
                || rows[0].Expires_At__c < now
                || rows[0].Last_Activity__c.addMinutes(idleMinutes) < now
                || rows[0].Portal_User__r.Status__c != 'Active'
                || rows[0].Family__r.Status__c != 'Active') {
            throw new PortalException('SESSION_EXPIRED');
        }
        Portal_Session__c s = rows[0];
        if (s.Last_Activity__c.addMinutes(1) < now) {
            s.Last_Activity__c = now;
            update s;
        }
        return new UserContext(s);
    }
}
```

Family-scoped selector

```
public without sharing class TransactionSelector extends FamilyScopedSelector {
    public TransactionSelector(UserContext ctx) {
        super(ctx);
    }

    public List<Transaction__c> forPeriod(Date fromDate, Date toDate) {
        Id familyId = ctx.familyId;
        Set<Id> memberIds = ctx.visibleMemberIds();
        return [
            SELECT Id, Txn_Date__c, Type__c, Amount__c, Direction__c, Payee__c,
                   Account__r.Name, Category__r.Name, Member__r.First_Name__c
            FROM Transaction__c
            WHERE Family__c = :familyId
              AND Member__c IN :memberIds
              AND Txn_Date__c >= :fromDate AND Txn_Date__c <= :toDate
            ORDER BY Txn_Date__c DESC
            LIMIT 200
        ];
    }
}
```

React client - one place for the token and expiry

```
import { createDataSDK } from '@salesforce/platform-sdk';

const TOKEN_KEY = 'vkf.session';

export async function vkf<T>(path: string, init: RequestInit = {}): Promise<T> {
  const sdk = await createDataSDK();
  const response = await sdk.fetch!(`/services/apexrest/vkf/v1${path}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      Accept: 'application/json',
      'X-VKF-Session': sessionStorage.getItem(TOKEN_KEY) ?? '',
      ...init.headers,
    },
  });
  if (response.status === 401) {
    sessionStorage.removeItem(TOKEN_KEY);
    window.dispatchEvent(new Event('vkf:session-expired'));   // AuthContext routes to /login
    throw new Error('SESSION_EXPIRED');
  }
  const body = response.status === 204 ? null : await response.json();
  if (!response.ok) throw new Error(body?.code ?? 'REQUEST_FAILED');
  return body as T;
}
```

18 Developer Edition constraints

Constraint (this org)	Impact	Design response
Data storage 5 MB (about 2,500 records)	Transactions, sessions and EMI rows fill it fast. Realistically 1–2 active families for a year or two.	Audit in a big object · rolling 12 EMI rows · 7-day session retention · notification purge · Monthly_Summary for history · storage widget for you · move to a paid org if you outgrow it
File storage 20 MB	Receipt photos	Phase 6 only; compress and cap at 300 KB each
15 single emails / day	Resets, reminders	In-app first; admin-assisted resets; one weekly digest
15,000 API requests / day	Every Apex REST call is assumed to count	One dashboard call per page view; no polling (idle-timer keep-alive at most every 10 min)
Governor limits (100 SOQL, 150 DML, 10 s CPU per transaction)	Bulk recalcs, hashing	Bulkified handlers; batch reconcile; aggregate queries; benchmark hash iterations
Licences: 40 External Apps Login + small community counts	Native users can't scale to unlimited families	Custom identity (this design) with the IdentityProvider seam
No Shield Platform Encryption	Only standard encryption at rest	Store minimal sensitive data (last 4 digits only); Classic Encrypted Text for the TOTP secret
Max 100 scheduled Apex jobs	Many schedules	7 orchestrating jobs; per-family work runs in Queueables

19 Testing and quality

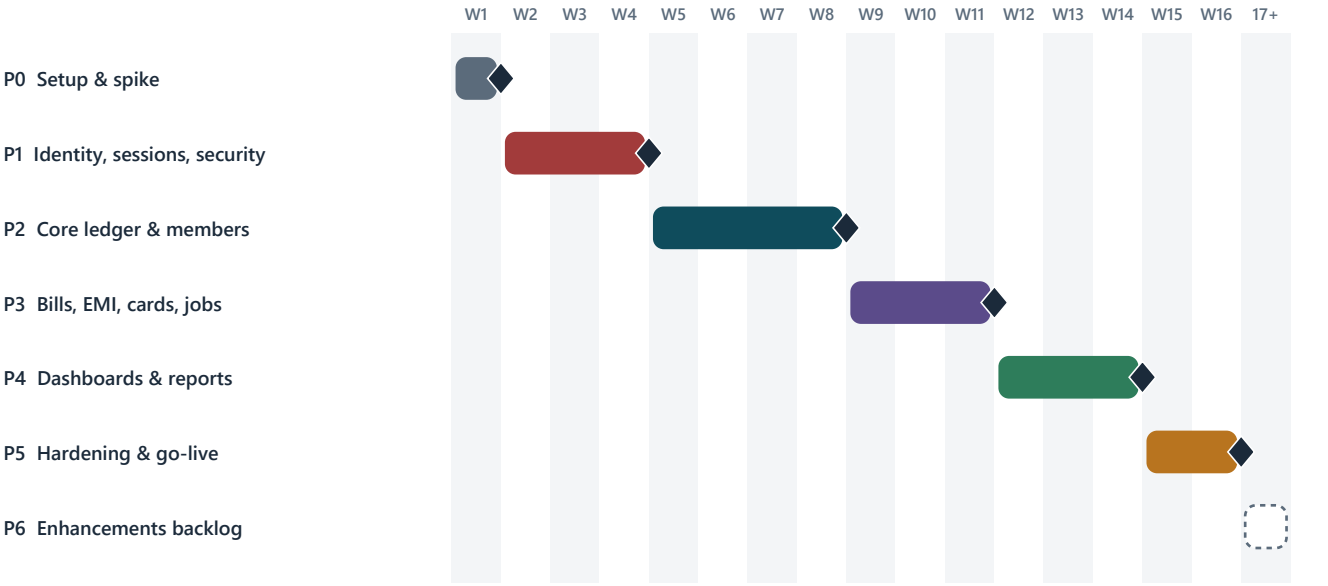
- **Apex unit tests** ≥ 85% coverage (deploys need 75%). `TestDataFactory` builds two families with every role.
- **Isolation suite** (sample below) runs for every REST resource and is a release gate.
- **Security tests:** lockout and throttle, expired and revoked tokens, idle timeout, reused reset token, member calling admin endpoints, mass assignment, injection strings in filters and sort fields.
- **Finance maths:** EMI matches known values (₹10,00,000 at 9% for 60 months = ₹20,758). Card cycles across month ends and leap years. Every transfer group nets to zero. Budget actuals stay correct on update, delete and undelete.
- **React:** Vitest + Testing Library for components and `vkfClient`, and Playwright end-to-end tests against the deployed site (both come with the template).
- **Local dev caveat:** `sf ui-bundle dev` proxies calls with *your* CLI credentials, not as the guest user. Guest restrictions are only exercised on the deployed site, so always run the isolation and security checks there.
- **Static analysis:** `sf code-analyzer run` (Apex + JS), ESLint, Prettier, and the Husky pre-commit hook already present in this repo.

Caller	Action	Target	Expected
Family A - Admin	Read transaction	Family A, any member	200 OK
Family A - Member	Read transaction	Family A, another member's private account	404 NOT_FOUND
Family A - Child	POST /family/members	-	403 NOT_AUTHORIZED
Family B - Admin	PATCH /transactions/{id}	Family A record Id	404 + audit CROSS_FAMILY_ATTEMPT
Family B - Admin	POST /transactions	Account Id from Family A	404 (lookup guard)
Family A - Member	POST /transactions with Family__c = Family B	-	Created in Family A (field ignored)
Family A - Co-Admin	Demote the Family Admin	Family A admin	403 NOT_AUTHORIZED
Anyone	Any endpoint	Expired, revoked or other-browser token	401 SESSION_EXPIRED

Definition of done (every story)

- Endpoint goes through SessionService, AccessGuard and FamilyScope; selector-only SOQL; DTOs whitelisted.
- Isolation and role tests added; coverage ≥ 85%; Code Analyzer shows no high-severity issues.
- Audit events emitted for every create, update, delete and security action; no secrets in debug logs.
- React page handles loading, empty and error states, and 401 is handled centrally.

20 Delivery roadmap



M0 W1 guest Apex REST works from React on the site · M1 W4 two families sign in, isolated · M2 W8 Family 1 runs a real month · M3 W11 dues remind themselves · M4 W14 admin reviews every member in < 2 min · M5 W16 v1.0 go-live

Figure 10 - 16-week plan for one developer, with milestones

Phase	Weeks	Deliverables	Exit criteria
P0 Setup and spike	1	Scaffold from reactexternalapp into this project · LWR site + guest permission set · `/vkf/v1/ping` Apex REST + React page · CI (lint, Prettier, Code Analyzer, Vitest) · CMDT types	**A guest browser calls Apex REST from React on the live site** with a custom header; pipeline green
P1 Identity and security core	2–4	Family, Portal_User, Session, Auth_Token, Password_History · Password, Session and Auth services · AccessGuard · FamilyScope · Audit · login, first-login, reset, change password · app shell + idle timer · onboarding flow · console app	Two families sign in; isolation tests pass; lockout and timeouts verified
P2 Core ledger and members	5–8	Accounts, categories, transactions, transfers, card usage and payments, balances, budgets, member management UI	Family 1 runs a real month; balances match the bank
P3 Commitments and automation	9–11	Bills and instances, loans and EMI schedule, card statements, recurring rules, scheduled jobs, notifications	Dues remind themselves for 2 weeks with no manual work
P4 Insights	12–14	Configurable dashboard, 13 reports, saved reports, CSV export, monthly summaries	Family Admin reviews every member's month in under 2 minutes
P5 Hardening and go-live	15–16	Admin MFA, throttling, security test pass, Health Check, retention jobs, user guide for the family	Pen-test checklist all pass; v1.0 tagged
P6 Enhancements	17+	Statement import, receipts, child spend requests, goals automation, PWA, insights	Prioritised backlog

21 Risks and mitigations

Risk	Likelihood / impact	Mitigation
A bug in custom auth exposes data	Medium / High	Single gateway, per-endpoint isolation tests, code-review checklist, Code Analyzer, security test pass before go-live
Guest + Apex REST from a React external app behaves differently than expected	Low–Medium / High	Phase 0 spike proves it on the live site in week 1; fallback is the same Apex behind an LWC/LWR portal (`@AuraEnabled` transport)
Developer Edition storage fills up	High / Medium	Big-object audit, retention jobs, rolling EMI rows, storage widget; plan a paid org beyond 2 active families
Salesforce tightens guest-user rules	Medium / High	Read each release's notes; data access is already Apex-only; the IdentityProvider seam allows a switch to licensed users
Multi-Framework is a young stack	Medium / Medium	Pin npm versions, follow template upgrades, keep business logic in Apex so the UI is replaceable
Licence compliance if moved to production	Low (Developer Edition) / High	Use Experience Cloud licences in any paid org
Data-entry fatigue - the family stops using it	High / High	Quick add, recurring rules, auto-pay posting, mobile-friendly UI, CSV import in Phase 6
Wrong balances	Medium / Medium	Ledger is the source of truth + nightly reconcile + Adjustment entries + double-entry tests
Scope creep	Medium / Medium	Phase gates; v1 scope frozen; Phase 6 backlog

22 Next steps - your first ten tasks

1. Initialise git for this project (it isn't a repository yet) and commit this blueprint under `docs/`, so every later step is reviewable.
2. Generate the scaffold in a temporary folder with `sf template generate project -n VKFinanceScaffold -t reactexternalapp`. Copy its `uiBundles`, `digitalExperiences`, `digitalExperienceConfigs`, `networks`, `sites` and guest permission set into this project, renamed to VKFinancePortal / VK_Finance.
3. Remove the template's native-login Apex (`UIBundleLogin`, `UIBundleRegistration`, ...) from the guest permission set, or keep it parked for option B. Turn guest Chatter off in the network file.
4. Phase 0 spike: a `VkfPingResource` (`/vkf/v1/ping`) that echoes the `X-VKF-Session` header, plus a React page calling it. Deploy and test in a private browser window as a guest.
5. Create the Custom Metadata types (Security_Policy, Role_Permission, Default_Category) and their default records.
6. Create Family__c, Portal_User__c, Portal_Session__c, Auth_Token__c and Password_History__c with the fields in section 7.
7. Build PasswordService with tests, and benchmark the iteration count in the org.
8. Build SessionService, AuthService and VkfAuthResource with tests (login, logout, session, first-login).
9. Build vkfClient, AuthContext, the Login and FirstLogin pages, the idle-timer dialog and the protected routes.
10. Build the FamilyIsolationTest harness and the onboarding Screen Flow **before** any finance object, so every later feature lands on a proven security base.

> **Sources checked for the React / Multi-Framework facts**
>
> Live org metadata and CLI help (27 Oct 2026): `UIBundle` type, `template generate ui-bundle`, `template generate project -t reactexternalapp`, `ui-bundle dev`. Generated reactexternalapp scaffold: VkfExtProbe (LWR app-container site, `@RestResource /auth/login`, guest API permission set). Salesforce Developers: "React Development Using Salesforce Multi-Framework" and the Multi-Framework overview; salesforcetrail.com, "React On Salesforce Is GA" (GA in Summer '26, Developer Edition support, data access through the Data SDK and Apex REST).